

Section 02



Industrial Security (Master)

Section 02 – PLC Ladder Logic Reverse Engineering

PLC Ladder Logic Reverse Engineering

Lecture Notes

Department of Computer Science

- 1 Why Ladder RE
- 2 Acquisition
- 3 Decoding
- 4 What to Look For
- 5 Defending the Code

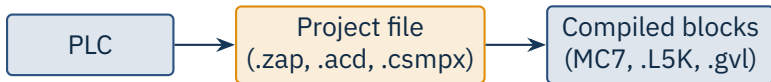
By the end of this section you will be able to

- Extract a ladder-logic program from a PLC and from a project file.
- Recognise vendor-specific block formats (Siemens, Rockwell, CODESYS).
- Decompile or visualise ladder logic and identify what the program does.
- Spot logic-bomb and stealth modifications (à la Stuxnet).
- Apply integrity controls to prevent unnoticed code change.

1

Why Ladder RE

Section 02 – PLC Ladder Logic Reverse Engineering



The blind spot

The PLC runs the *compiled* blocks. The vendor tool shows the *project*. If an attacker changes only the runtime, the project still looks fine and only RE catches it.

Source: Stuxnet (2010) is the canonical example – HMI fed clean data while the PLC ran tampered logic.

	Siemens S7-3xx/4xx	Rockwell ControlLogix	CODESYS family
Project file	.zap, .s7p (Step7); .ap16 (TIA)	.acd	.project (XML zip)
On-PLC format	MC7 byte-code, OBxx/FCxx/DBxx blocks	.L5K compiled tags	UDFB structured text
Read-back tool	snap7, TIA upload	RSLogix5000 upload, libplctag	CODESYS device tree

Source: Snap7 docs; libplctag; Rockwell PCDC; CODESYS GmbH.

2

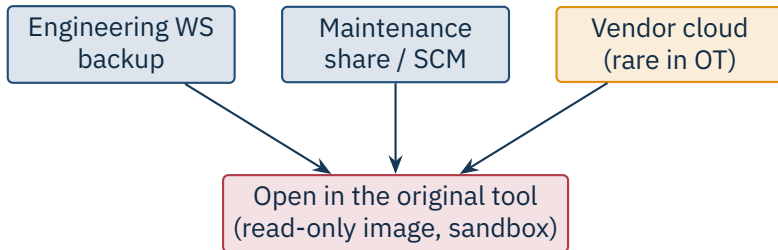
Acquisition

Section 02 – PLC Ladder Logic Reverse Engineering

```
1 import snap7
2 c = snap7.client.Client()
3 c.connect("10.0.0.10", 0, 1)
4 # List blocks, then upload OB1 + DB1
5 blocks = c.list_blocks() # SfcCount, OBCount, FCCount...
6 ob1 = c.upload(snap7.types.Block.OB, 1) # raw MC7 bytes
7 db1 = c.upload(snap7.types.Block.DB, 1)
8 open("ob1.bin", "wb").write(ob1)
```

Permission check

The S7-300/400 default *put/get* setting allows upload; modern S7-1500 requires explicit enable. Document any change.



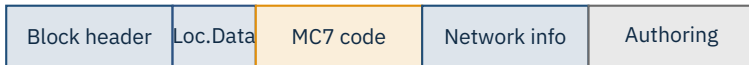
Always sandbox

Project files have run macros and *network ranges*. Open them in a disconnected VM with no host shares.

3

Decoding

Section 02 – PLC Ladder Logic Reverse Engineering



Tools

s7-pcap and step7-parser (Sergey Gordeychik et al., 2015) parse MC7. Newer S7commPlus blocks (S7-1200/1500) use Pdata and require S7commPlus-aware tools.

Source: Sergey Gordeychik et al., SCADA StrangeLove, 2015 | Beresford, D., Exploiting Siemens Simatic S7 PLCs, 2011.

- .acd is a binary container; .l5k / .l5x is the printable export.
- l5k_parser (community) reconstructs Structured Text from the export.
- Watch for **add-on instructions (AOIs)** – compiled inside the project; their internals are hidden in the GUI.
- Tag aliases obscure semantics: Bool115 may secretly be “open_valve”.

Decompile then re-render

Round-trip the export through RSLogix; what visually matches the original is faithful, what diverges is your target.



CODE16 (2023)

Sixteen CVEs in CODESYS Runtime exposed authentication, parser, and update-verifier flaws across hundreds of vendor products built on the runtime.

Source: CISA ICSA-23-103-09 and related advisories; Microsoft CoDe16 disclosure, 2023.

4

What to Look For

Section 02 – PLC Ladder Logic Reverse Engineering

1. Pristine OB1 / main loop sets up normal control

2. Hidden OB35 (cyclic) checks for the target signature (frequency, set-point)

3. On match: record current set-points, replace with malicious sequence

4. When operator reads back, recorded set-points are returned (HMI lied)

Source: Langner, R., To Kill a Centrifuge, The Langner Group, 2013.

- Unused inputs that are read but never appear in the rung logic.
- Time- or counter-based branches that never fire under normal data.
- Blocks named after vendor utilities (“OEM_diag”, “factory_test”) outside the official block list.
- Constants whose values match *exactly* known process set-points.
- Calls into reserved or vendor-locked memory areas.
- Discrepancy between project-file timestamps and on-PLC block CRCs.

Caution

A missing CRC, or a CRC that does not match the project file, is more often vendor sloppiness than malice – but it deserves a written investigation.



Reverse engineering tip

Cross-reference *every* symbol against where it is read and where it is written. Symbols that are only *written* (never read) are smoke.

5

Defending the Code

Section 02 – PLC Ladder Logic Reverse Engineering

Signed firmware + signed program downloads (vendor support varies)

PLC key-switch in “RUN” position; “PROG” requires physical access

Continuous block-checksum monitoring (Claroty, Dragos, Nozomi modules)

Version-controlled project files with CI & review

Engineering workstation hardened; no Internet, MFA, application allow-listing

- Findings on production logic touch **safety** – coordinate with process safety, not just IT.
- Vendor PSIRT first; CERT/CC second; public publication only after fix.
- Sharing tags or constants from real plants risks identifying the asset owner – redact heavily.
- Some jurisdictions (US DMCA §1201, EU CRA research safe-harbour, German §202c StGB) constrain RE; consult counsel.

Caution

Logic from a production PLC you do not own is contaminated evidence. No serious vendor will accept it.

★ Key Takeaway: Ladder RE for ICS

- PLCs run compiled blocks, not the project source – they can diverge silently.
- Project-file and runtime acquisition each catch different attacks.
- Vendor-specific tools (snap7, l5k-parser, CODESYS) are the workhorses.
- Stuxnet-style logic bombs hide in unused inputs, time bombs, and CRC mismatches.
- Defence is signed downloads, key-switch, continuous checksum monitoring.

- Langner, R. *To Kill a Centrifuge*, The Langner Group, Nov 2013.
- Falliere, N., Murchu, L., Chien, E. *W32.Stuxnet Dossier*, v1.4, Symantec, 2011.
- Beresford, D. *Exploiting Siemens Simatic S7 PLCs*, BlackHat USA, 2011.
- Gordeychik, S. et al. *SCADA StrangeLove* talks and tools, 2014–2018.
- Snap7 project. <https://snap7.sourceforge.net>.
- OpenPLC project. <https://openplcproject.com>.
- ODVA. *libplctag / CIP open libraries*.
- Rockwell Automation. *Logix5000 Controllers Import/Export*, manual 1756-RM084.
- CISA ICSA-23-103-09 and related CODESYS advisories (CoDe16), 2023.
- Microsoft Threat Intelligence. *CoDe16: 16 vulnerabilities in CODESYS Runtime*, 2023.
- IEC 61131-3:2013. *Programmable controllers – Part 3: Programming languages*.
- Dragos, Claroty, Nozomi. PLC integrity-monitoring product documentation.

End of Section 02

PLC Ladder Logic Reverse Engineering

Questions and discussion